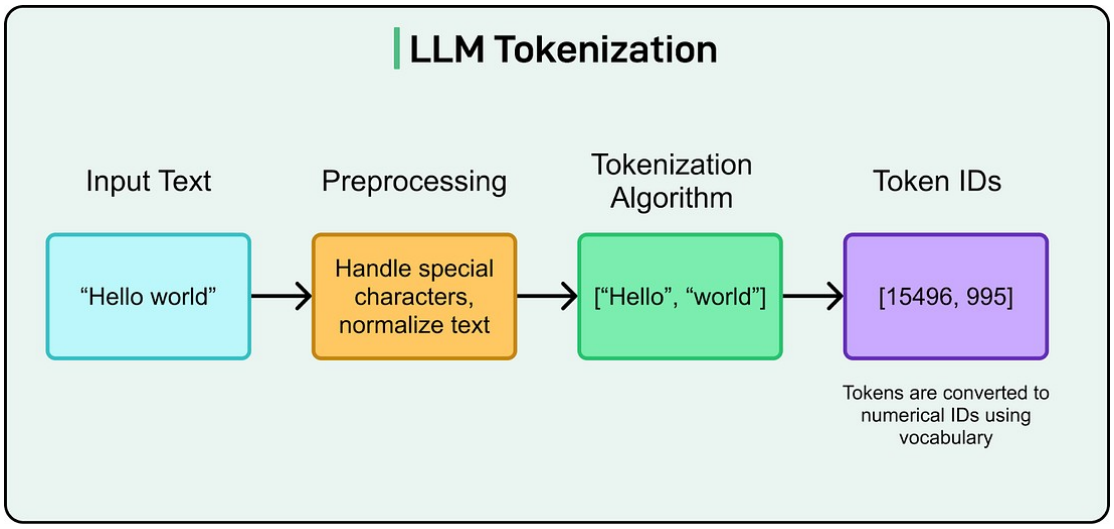
For LLMs, tokens are the fundamental units of text that the model processes. When you type 'Hello world!' to ChatGPT, it doesn't see two words and punctuation, but rather it sees perhaps four distinct tokens: ['Hello', ' world', '!', '\n'].

Tokens are what rule the world of LLMs. You send tokens to models, you pay by the token, and models read, understand, and breathe tokens.



What Are Tokens?

Tokens are the fundamental units of text that an LLM processes.

However, tokens are **not** always equivalent to words. Depending on the tokenization approach used, a token could represent:

- A single character
- A subword (part of a word)
- A complete word
- Punctuation marks
- Special symbols
- Whitespace characters

For example, the sentence "I love machine learning!" might be tokenized as: ["I", "love", "machine", "learning", "!"] or ["I", " love", " machine", " learn", "ing", "!"] depending on the tokenization method.

Why Tokenization Matters

Tokenization is very important for several reasons:

1. **Vocabulary Management**: LLMs have finite vocabularies (typically between 30K-100K tokens). Tokenization allows these finite vocabularies to express an open-ended language space. By breaking rare or complex words into reusable subword units (e.g., "extraordinary" → "extra" + "ordinary"), the model avoids needing a separate token for every possible word in every language.

2. **Handling Unknown Words**: Good tokenization strategies can break down unfamiliar words into familiar subword units, enabling models to handle words they've never seen before. For example, a model that has never seen the word “biocatalyst” might still recognize “bio” and “catalyst” as separate tokens and draw useful meaning from them.

3. **Efficiency**: The length of text sequences directly impacts computational requirements. Efficient tokenization reduces the number of tokens needed to represent text.

4. **Model Performance**: The quality of tokenization affects how well LLMs understand and generate text, especially for non-English languages or specialized domains. Poorly tokenized input can fragment meaning or distort structure.

How Tokens are Read By LLMs

Once text is tokenized, there is one more step that transforms these symbolic tokens into something the neural network can actually process: numerical representation. Each token in the vocabulary is assigned a unique integer ID (called a token ID). For example.

"Hello" → token ID 15496

" world" → token ID 995

These token IDs are then converted into high-dimensional numerical vectors called embeddings through an embedding layer. Each token ID maps to a dense vector of real numbers (typically 512, 1024, or more dimensions). For instance, the token "Hello" might become a vector like [0.23, -0.45, 0.78, ...].

This numerical transformation is necessary because neural networks can only perform mathematical operations on numbers, not on text symbols. The embedding vectors capture semantic relationships between tokens, where similar tokens have similar vector representations in this high-dimensional space. This is how models "understand" that "king" and "queen" are related, or that "run" and "running" share meaning.

Common Tokenization Methods

1. Byte Pair Encoding (BPE)

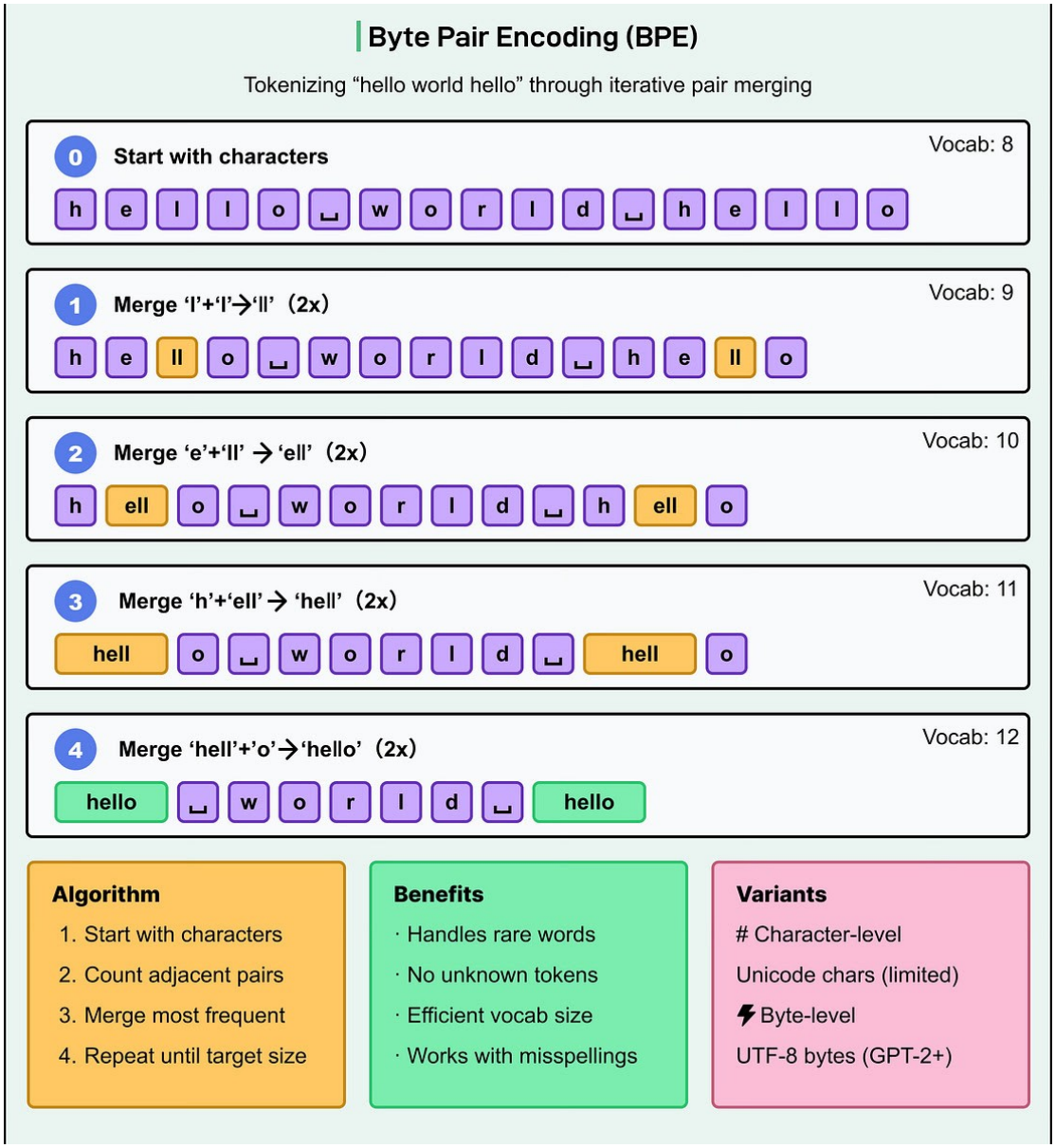
BPE is one of the most widely used tokenization methods in modern LLMs, used by models like GPT-2, GPT-3, and GPT-4.

How it works:

- Start with a basic vocabulary containing individual characters
- Count the frequency of adjacent character pairs in the training corpus
- Iteratively merge the most frequent pairs into new tokens
- Continue this process until reaching a desired vocabulary size

BPE creates a flexible subword vocabulary that efficiently represents common words while still being able to break down rare ones. This helps models handle misspellings, compound words, and unknown terms without resorting to an "unknown token."

A key variant is **byte-level BPE**, which works directly with UTF-8 bytes rather than Unicode characters. This makes sure that any possible character can be represented (even those not seen during training), avoiding the "unknown token" problem.



2. WordPiece

WordPiece was introduced by Google and is used in models like BERT, DistilBERT, and Electra.

How it works:

- Similar to BPE, it starts with a character-level vocabulary
- Instead of merging based purely on frequency, it selects pairs that maximize the likelihood of the training data
- It uses a special prefix (typically "##") to mark subword tokens that don't start a word

For example, "unhappy" might be tokenized as ["un", "##happy"] in WordPiece.

3. SentencePiece

SentencePiece is a tokenizer developed by Google that works directly on raw text without requiring language-specific pre-tokenization. It’s used in models like T5, XLNet, and ALBERT.

How it works:

- Treats input text as a raw stream of Unicode characters, including spaces
- Space is preserved as a special symbol (typically "▁")
- Can implement either BPE or Unigram Language Model algorithms
- Eliminates the need for language-specific pre-tokenization
- Particularly effective for languages without explicit word boundaries (like Japanese or Chinese)

For example, the phrase "Hello world" might be tokenized as ["▁Hello", "▁world"], where the ▁ indicates a word boundary.

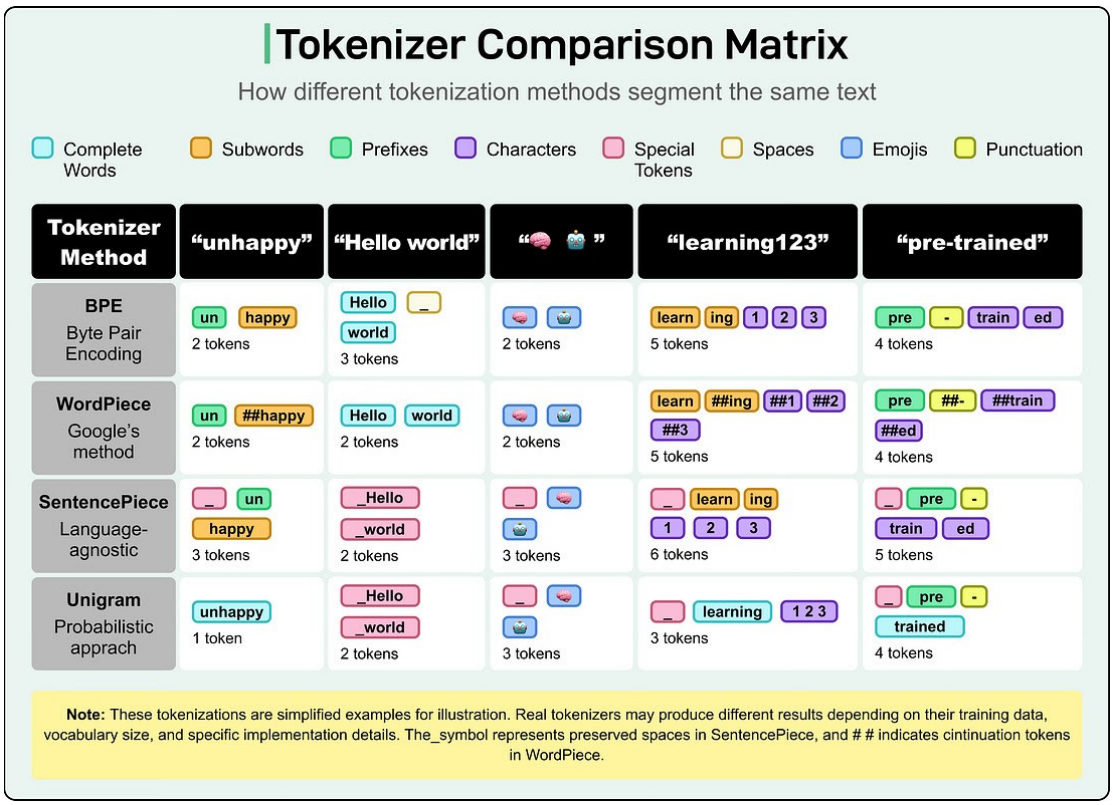
4. Unigram

Unigram is often used together with SentencePiece and takes a probabilistic approach rather than a merge-based one.

How it works:

- Start with a large vocabulary of possible subword candidates
- Iteratively remove tokens that least affect the model's ability to represent the training data
- Uses a probabilistic model to find the most likely segmentation of words

Unlike BPE or WordPiece, which build their vocabulary by merging, Unigram works more like sculpting, starting big and pruning down. This allows it to maintain a broader set of tokenization options and gives it more flexibility during inference.



Tokens and Context Windows

LLMs have a limited "context window," which is the maximum number of tokens they can process at once. This limit directly affects:

- **Input length**: How much text the model can consider before generating a response.
- **Output length**: How much can it generate in a single completion.
- **Coherence**: How well it can maintain topic consistency across longer conversations or documents.

Older models like GPT-2 were limited to ~1,024 tokens. GPT-3 increased this to 2,048. Today, cutting-edge models have limits of **1M+**, such as Gemini 2.5 Pro.

What To Know About Tokenization

Token Counting

Understanding token counts is important for:

- Estimating API costs (many LLM APIs charge per token)

- Staying within the context window limits
- Optimizing prompt design

As a rough estimate for English text (this varies!):

- 1 token ≈ 4 characters
- 1 token ≈ 0.75 words
- 100 tokens ≈ 75 words, or ~1 paragraph

Tokenization Quirks

Tokenization can lead to some unexpected behaviors:

1. **Non-English Languages**: Many LLMs tokenize non-English text inefficiently, using more tokens per word than English.
2. **Special Characters**: Unusual characters, emojis, or specific formatting might consume more tokens than expected. For instance, a single emoji like "  " might consume several tokens, depending on the tokenizer, which can distort meaning or unexpectedly bloat token usage.
3. **Numbers and Code**: Some tokenizers handle numbers and programming code in counter-intuitive ways, breaking them into multiple tokens. This fragmentation makes it harder for models to reason numerically or generate accurate code, since the logical or mathematical unit is not preserved as a whole.

How Tokenization Impacts LLM Performance

Many challenges and quirks in large language models stem not from the model itself, but from how text is tokenized. Here's how tokenization affects different areas of performance:

- **Spelling and Typos**: When a user misspells a word, tokenizers often break it into unfamiliar or rarely seen token combinations. Since the model learns patterns from frequent token sequences, unfamiliar tokens disrupt its ability to understand or correct the input.
- **Cross-Language Performance**: Tokenizers designed primarily for English tend to break words in other languages—especially those with different scripts or morphology—into more tokens. This leads to higher token counts, poorer context compression, and reduced fluency and accuracy in non-English text.

- **Numerical and Mathematical Reasoning**: Numbers are often split into multiple tokens (e.g., "123.45" might become ["123", ".", "45"]). This disrupts numerical understanding and arithmetic operations, since the model doesn't always see numbers as atomic units but rather as sequences of parts.
- **Code Generation and Understanding**: Programming languages rely on precise syntax and structure. Tokenizers that break up operators, identifiers, or indentation inconsistently can hinder a model's ability to generate or interpret code correctly. Well-aligned code tokenization improves model accuracy for completions, formatting, and bug detection.

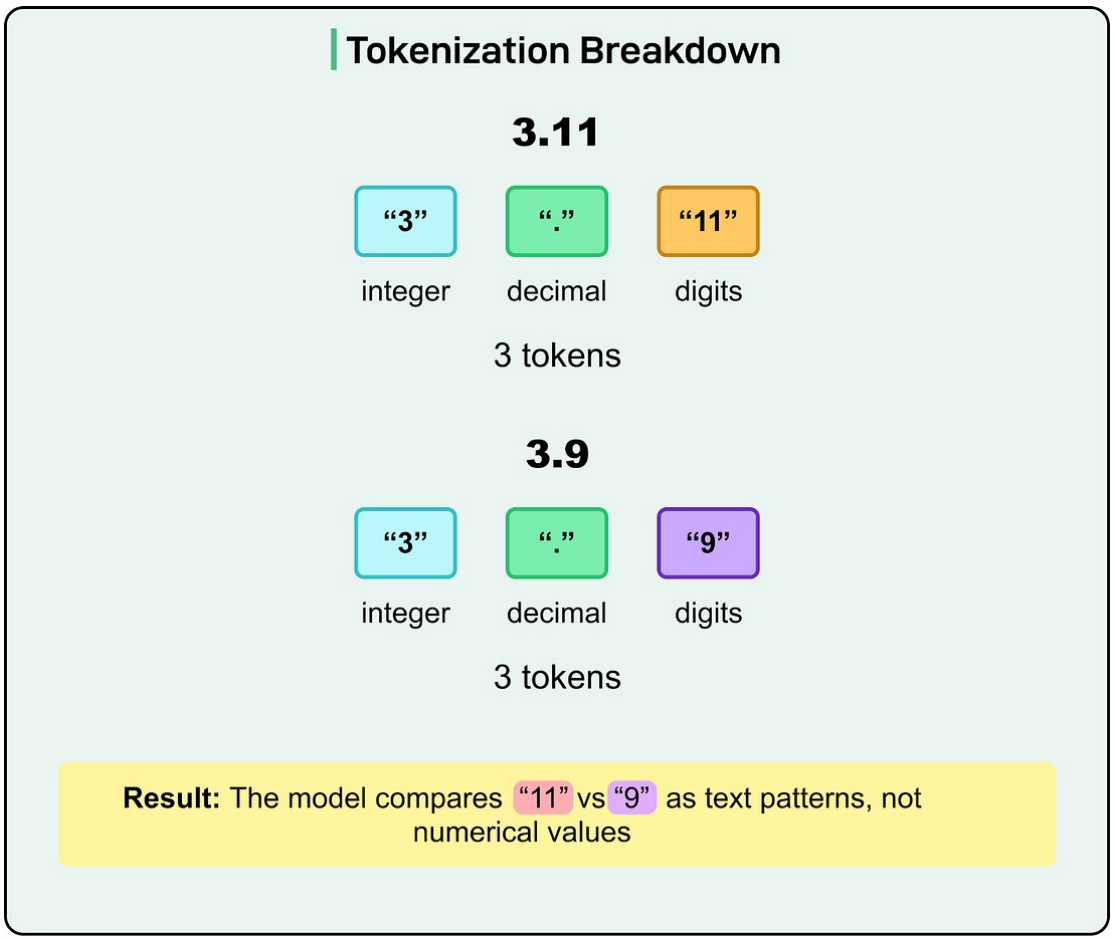
The Infamous 3.11 vs 3.9 Problem

Large language models often fail at seemingly simple numerical comparisons like “What is bigger: 3.11 or 3.9?”. Tokenization provides insight into how numbers are processed under the hood.

Let's look at these numbers: 3.11 and 3.9. When tokenized, these are broken into separate components. For simplicity, let's say that “3.11” is split into tokens like "3", ".", and "11", while “3.9” is split into "3", ".", and "9". To a language model, these aren’t numerical values but symbolic fragments. The model isn’t comparing 3.11 against 3.9 as floating-point values. It’s pattern-matching based on the statistical likelihood of what text should come next, given how it has seen these tokens appear in its training data.

There are multiple ways for models today to answer these correctly:

- They may just answer it correctly through sheer randomness. Since LLMs are inherently non-deterministic, sometimes the answer is right, sometimes it's wrong.
- LLMs can be trained with specific SFT training data or have system prompt instructions when comparing popular "tests" like these. This is similar to hard-coding the right answer.
- LLMs today often are not alone. They have access to **tools** that help them do math, data analysis, and web searches. These tools allow the model to compare these discrete numbers correctly.
- Sometimes, the prompt framing and context do matter. For example, prompts that hint toward quantitative reasoning may perform better than generic, unclear prompts.



Conclusion

Tokenization is how LLMs break down text into processable units before converting them to numbers. Text like "Hello world!" becomes tokens like ['Hello', ' world', '!'], then gets converted to numerical vectors that neural networks can understand. Common methods include BPE (used by GPT models), WordPiece (BERT), and SentencePiece (T5).

Tokenization directly impacts costs (you pay per token), context limits (models can only process so many tokens), and performance quirks. It explains why LLMs struggle with math (numbers get split up), why non-English text is less efficient (more tokens needed), and why models fail at "3.11 vs 3.9" comparisons (they see fragmented symbols, not numbers).

Understanding tokenization helps you write better prompts, estimate API costs, troubleshoot issues, and grasp both the capabilities and fundamental limitations of modern AI. It gives you deeper insight into both the capabilities and limitations of modern AI, as it's the lens through which LLMs see everything.